P1368R1: Multiplication and division of fixed-point numbers

Audience: SG6
S. Davis Herring <herring@lanl.gov>
Los Alamos National Laboratory
March 5, 2019

Introduction

SG6 discussed John McFarlane's P0037R5 (since updated in response to the first version of this paper) on Friday in San Diego, and my realization was that much of the difficulty in specifying a fixed-point number type hinges on identifying the "natural" behavior for multiplication and division. The group asked me to write about the different possibilities; the hope is that specific choices can then be made to increase the focus of future development.

The most obvious choice of type for a homogeneous binary operation (in the absence of special auto-widening) is that of the inputs. (For a C++ fixed-point type, we expect the type to include the allocation of its digits to integer and fractional parts.) This choice is subject to overflow and imprecision: 12.5×23.6 is 295 (which is too big for the *dd.f* format of the inputs), and 01.3×02.4 is 3.12 (which lies between *dd.f* values). Nonetheless, for fixed-width arithmetic it might seem to be a reasonable *a priori* default.

Note that fixed-point formats can include leading or trailing zeroes. For example, *0.0fff* or *ddd00* store only three digits each; they can be said to have *negative* numbers of integer and fractional digits, respectively. The example values below avoid such formats (and use decimal digits) for clarity.

History

Since R0:

- Mentioned P0037R6
- Discussed the modulo operation
- Explained that P0554R0's division choice is outdated (by P0037R4)
- Mentioned P1050 as an alternative to `/`
- Discussed `fixed_point` *vs.* `scaled_integer`

Underlying integer operations

In Jacksonville, I asked John about the choice made by his `fixed_point` template: in particular, why the multiplication of two values of format *dddd.ff* (with the same width of result) was *dd.ffff* rather than rounding to fit back into the same *dddd.ff* format. In particular, I was bothered by the increased risk of overflow from the smaller maximum value (e.g., even 03.5×03.2 overflows the resulting *d.ff*). He explained that the multiplication **is** multiplication of the underlying integers, along with a separate (static) operation of adjusting the exponents; it therefore shares the underlying integer multiplication's properties of being exact and overflow-prone in the absence of widening.

P0037R5 says that `fixed_point` division is analogously defined as division of the underlying integers with a separate exponent selection: dividing *dd.fff* by *d.f* gives *ddd.ff*. (For example, 12.345 divided by 0.1 is 123.45.) Continuing the analogy with integer operations, this truncates and cannot overflow for a same-sized result (disregarding −1 and 0 divisors); this necessitates the increased maximum value of the result (when there are any fractional digits in the divisor). Despite these familiar properties, the loss of precision can be as surprising as the overflow from multiplication: 5.2163/1.6604 is just 3, because 5.2163/0.0003 needs to be 17387 (and so the 3 is actually 00003). More generally, dividing any format by itself yields an integer.

Division of decimal IEEE 754 numbers, where the exponent is (partially) independent of the value, affords an interesting comparison here; it *starts* with the `fixed_point` rule, but shifts more digits into the significand as needed to avoid (or minimize) rounding. For a type with a static exponent, widening is of course the only means of adding those additional digits.

P0037R5 continues the pattern with the remainder operation: *dd.fff* modulo *ddd.f* gives *d.fff*. (For example, 12.345 modulo 500.0 is 2.345.) This operation cannot overflow; the result has the width of the narrower operand with the fractional digits of the dividend. As with integer operations, the dividend can be recovered exactly from the quotient and remainder. However, the relationship between the remainder and divisor pertains to the integer values; the range of values for the `fixed_point` remainder may be unrelated, and the result is unrelated to divisibility. (For example, 12.3 modulo 0.025 is 02.3.)

Of course, C and C++ do not define a remainder operation for floating-point numbers at all. Nonetheless, there is a suitable generalization of Euclidean division to real numbers that retains the quotient (but not the remainder) as an integer; this is the operation performed by `std::fmod` and by `%` in Java. It is particularly well-suited to floating-point arithmetic in that it is always exact.

Another complication is negative operands: there are various conventions for the integer result of `/` and for the range of `%`. For example, Python defines `//` as the *floor* (not the truncation) of the real-number result and (thus) gives `%` the sign of the *divisor*. The result has a better claim on the term *modulo*, but is inexact for floating-point types: in particular, it can equal the divisor. Yet another choice is that of `std::remainder` (the only remainder operation defined by IEEE 754). It is exact; its result range is centered on 0 regardless of the signs of the operands. (Python also supplies `math.fmod` and (since 3.7) `math.remainder`.)

Precise division

There are two strategies for getting more fractional digits in a quotient: shift the divisor right, reducing its precision (5.2163/0001.6 is 03.260) or shift the dividend left, risking overflow (001.00/000.07 is 00014, but 1.0000/000.07 is 014.28). The overflow from shifting corresponds to that from multiplication; it may of course be avoided by increasing the width of the dividend by the shift distance (5.216300000/1.6604 is 00003.14159). The division never overflows (as above), but narrowing back to the original format might (2.718300000/0.2327 is 00011.68156, which doesn't fit back into *d.ffff*).

Lawrence Crowl in P0106R0 points out that shifting the dividend left by (at least) the width of the divisor guarantees that no quotient rounds (or even truncates) to zero. (There is a typo in the paper; the resolution parameter is supposed to be @a-$b. John at one point considered the same behavior for `fixed_point`.) We may call this "quasi-exact division"; it is analogous to multiplication in that the width of the result is the sum of those of the operands. Since this shift distance is known *statically*, it can be used to select a wider integer type (assuming a suitable trait for the width of the divisor). (Shifting by *twice* the width of the divisor guarantees that any change to the dividend or divisor (but not both) changes the result.)

P0106R0 defines `%` only for its integer types. It can of course be defined in terms of the same shift as for quasi-exact division; then dividing 7 by 3 produces 2.3 r .1. While the dividend can be reconstructed (with trailing 0s) from this sort of quotient and remainder, the latter has the conventional meaning for neither integers nor real numbers.

By shifting to give the dividend and divisor the same exponent before performing Euclidean division, the dividend reconstruction property can coexist with the expected range for the remainder. (For example, 1 divided by 0.07 is then 14 r 0.02.) However, the resulting quotient is of course always an integer; worse, its width depends on the operand *exponents*. In particular, it is simply 0 if the divisor's exponent is much larger than the dividend's and is very wide if the opposite relationship holds.

The remaining alternative is to define `%` separately, following its definition for real numbers. It can be computed exactly in all cases with a result no wider than the dividend (and narrower if its most-significant digit is more sigificant than that of the divisor). When the dividend exponent is much larger than the divisor's, repeated scaled subtraction or modular exponentiation may be used to compute the contribution from the (notional) shift.

Width reduction

It is then a basic issue for multiplication and division that an "exact" result without overflow requires doubling the width. Applications with statically bounded expression depths can use width-tracking class templates (*e.g.*, `integral` or `elastic_integer`) to statically preclude overflow and imprecision with fixed-width (possibly primitive) underlying types. (Note that division cannot reduce the dividend's width because the divisor might be 1.) However, any loop with an accumulator absolutely must reduce the width of its results, as must anything with a large expression depth to avoid the memory and time cost of over-wide arithmetic.

Whether such reduction should discard the most significant digits (risking overflow), the least significant digits (risking imprecision), or some of each is of course application-specific. (*E.g.*, when dividing 12.345 by 1.1, discarding one of each to get the format (*d*)*dd.fff*(*f*) best captures the result as 11.222.) In the absence of shifting, `int` discards high digits for multiplication and low digits for division; P0037R5 does the same, or with an `elastic_integer` representation type discards none for multiplication.

When to shift

The P0037R5 approach follows "don't pay for what you don't use" by performing division without any shifts. However, shifts may still happen later:

```
template<class T>   // C++03 style
T normalize(T a,T b,T x) {
  return (x-a)/(b-a);
}
auto f() {
  using fix=fixed_point<int,-16>;
  return normalize(fix(1.25),fix(2),fix(1.5));
}
```

Such shifting after division is pernicious: this `f` returns what should be 1/3 with 16 fractional bits, but all are zeroes shifted in by the conversion in `normalize`.

Even when producing a widened result, initial shifting (as used by P0106R0's `nonnegative`/`negatable`) is necessary to avoid such behavior. Width reduction requires shifting the results of both multiplication and division. Width-reduced division may nonetheless be performed with *one* shift because dividend digits never influence more-significant quotient digits; trailing quotient digits may be *prevented* by reducing the precision shift by the same number of digits. (If the left shift is performed with *signed* arithmetic, an optimizer could notionally take advantage of undefined behavior for overflow to combine these.)

The result type of `/`

It is useful to parametrize code over types that act like integers (including integers wider than a register or with dynamic width, integer wrappers that prevent undefined behavior, and integers whose types carry additional static data). It is also reasonable to write code parametrized over types that approximate real numbers (including the various floating-point types, exact rationals, and fixed-point numbers). It appears impractical to choose a result from division that works with both of these kinds of generic code.

Generic code cannot decorate operands to guide the widening or shifting for an operator and prevent surprising results like 1.5/0.25 being zero (**tens**). Neither can it perform conversions on the operator's result, except perhaps back to an input type (which requires shifting to avoid false precision). In particular, the likelihood of code like

```
auto score(auto a,auto b,auto x,auto y) {
  return (a*x+b*y)/(x*x+y*y);
}
```

argues against any approach based on expression templates (especially those containing *references* to their operands). It also cannot profit from tools like `fractional` from John's P1050R0 that use the target type of a conversion to inform the execution of division.

The only other means of customizing an operator is to provide alternative implementations in namespaces; this too is unusable in generic code (and precludes having a default at all). The most broadly useful value-type default for each operator must therefore be chosen. For division of (approximate) real numbers, the quasi-exact precision is the obvious candidate: it is usefully consistent to have multiplication and division widen their results identically.

Sharp edges

The requisite shift may overflow a fixed-width (perhaps primitive) representation type for fixed-point numbers. Support for such underlying types is a deliberate feature of `fixed_point`, but may prove too dangerous. One could make the shifting behavior depend on the underlying integer type, but the inconsistency would be hard to teach.

Note that far more than the quasi-exact precision is wanted in certain cases (*e.g.*, a pre-computed reciprocal of a small integer with 32-bit accuracy). Users of a generic function that might perform such a division must pad the dividend on the right (or, with automatic widening, the divisor on the left) with zeroes to avoid silent precision loss.

Conclusion

The shifting recommended by P0106R0 but not P0037R5 has a non-zero cost and may preclude the direct use of primitive representation types. For real-number arithmetic, it is nonetheless the best default in an environment that relies very heavily on defaults.

Since the shifting is not always desirable (and is inconsistent with useful definitions for `%`), the separation of `scaled_integer` and `fixed_point` considered in P0037R6 is an attractive option. In addition to quasi-exact division, `fixed_point` would be a sensible place to include the exact real-number remainder operation. It may also be useful to provide convenient function templates that determine and apply the necessary adjustments to obtain a given precision or avoid shifting.

Acknowledgments

Thanks to Hubert Tong for suggesting the comparison to decimal floating-point arithmetic. Thanks to Matthias Kretz for pointing out that shifting for precision could be widening. Thanks to Rachel Ertl for helping derive

conclusion from observation. Thanks to John McFarlane for extensive discussion of multiple numeric types.